

Finding Forks

CPU TIME LIMIT

1 second

MEMORY LIMIT

1024 MB

DIFFICULTY

easy (1.4)

Your cutlery drawer contains many types of forks. Each with their own purpose, and each with its own place in the cutlery drawer. After a nice dining party with all your friends, disaster struck! You put all the used forks in the dishwasher, but now you are unsure where to put back some of the forks, because at least two places in the cutlery drawer are empty! And worse, you do not remember which type of fork belongs where!



What is the minimum number of forks that must have been in the dishwasher to cause this confusion?

A small selection of the many types of forks that you own. CC BY-SA 3.0

by Mark Taff on [Wikimedia](#)

[Commons](#)

Input

The input consists of:

- One line with an integer n ($2 \leq n \leq 10^5$), the number of types of forks.
- One line with n integers a ($1 \leq a \leq 10^9$), the number of forks of each type.

Output

Output the minimum number of forks that must have been in the dishwasher.



Sample Input 1

```
3
4 9 5
```

Sample Output 1

```
9
```

Sample Input 2

```
10
18 39 5 12 1000000000 54 23 11 123
31415
```

Sample Output 2

```
16
```

Eeny Meeny

CPU TIME LIMIT

2 seconds

MEMORY LIMIT

1024 MB

DIFFICULTY

easy (1.8)

“Eeny meeny miny moe” is a well-known nursery rhyme in English, used (among other things) by kids to “randomly” select members of a team. It exists in many variations, one of which goes like this:

*Eeny, meeny, miny, moe,
Catch a tiger by the toe.
If he hollers, let him go,
Eeny, meeny, miny, moe.*



*Picture by Gina Harbach Glenn Green on
Flickr, cc by-nd*

Similar verses exist in most languages, such as “Ulle dulle dof” in Finnish, “Akka bakka bonka rakka” in Norwegian, and “Ole dole doff” in Swedish.

Two teams are to be selected for a game and the rhyme is used to select one kid for a team at a time, alternating between the two teams, until all kids have been selected. The kids are standing in a circle. In each selection round we start counting the kids in clockwise order around the circle, skipping one kid for every word in the rhyme, until the last word. The kid matching the last word is chosen for the current team and then the next round starts. In all rounds but the first, the counting starts at the next remaining kid (in clockwise order) after the one that was selected in the previous round. See Figure 1 for an example.

Given such a rhyme, and a group of kids, can you tell which kids will be in which team?

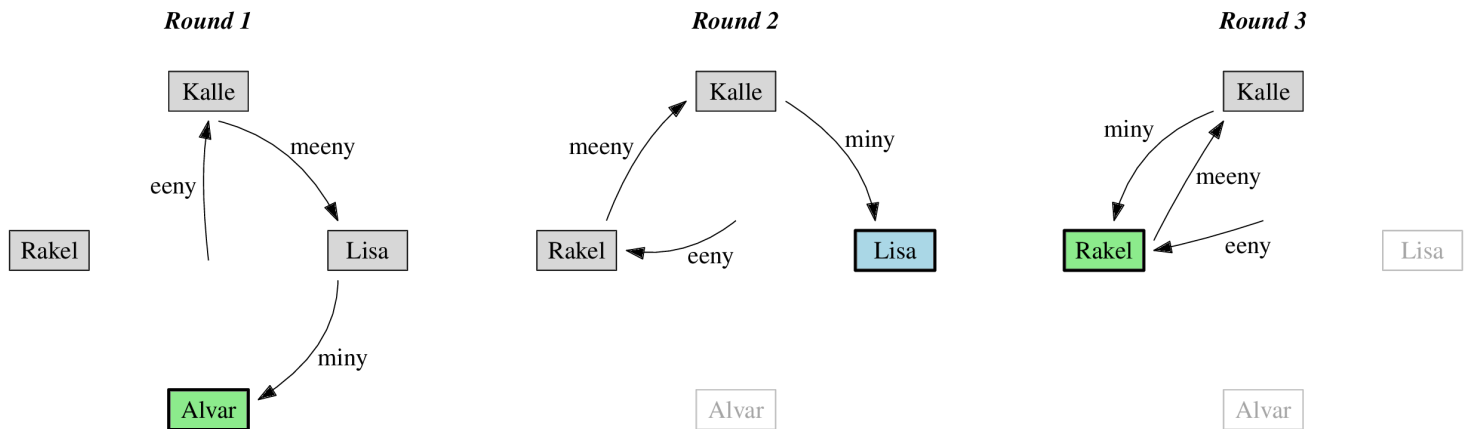


Figure 1: Illustration of the first three rounds of Sample Input 1. In rounds 1 and 3, Alvar and Rakel get selected for the first team, and in round 2, Lisa is selected for the second team. In round 4 (not shown), only Kalle remains and is selected for the second team.

Input

The first line of input contains the rhyme, consisting of a list of words separated by spaces. The second line of input contains an integer n ($1 \leq n \leq 100$), the number of kids. Then follow the names of the kids, one per line. The kids are given in clockwise order and the first kid listed is the one at which counting starts in the first round.

All words and names consist only of upper and lower case letters 'A'-'Z' and 'a'-'z'. No input line is empty or longer than 100 characters (excluding the newline character at the end of the line).

Output

Output the two teams, starting with the one whose first member is chosen first. For each team, output the number of kids in the team, followed by the names of the kids in the team, in the same order as they were chosen for the team.



Sample Input 1

```
eeny meeny miny
4
Kalle
Lisa
Alvar
Rakel
```

Sample Output 1

```
2
Alvar
Rakel
2
Lisa
Kalle
```

Sample Input 2

```
Every Other
3
a
b
c
```

Sample Output 2

```
2
b
c
1
a
```

Toilet Seat

CPU TIME LIMIT

1 second

MEMORY LIMIT

1024 MB

DIFFICULTY

easy (2.5)

Many potential conflicts lurk in the workplace and one of the most sensitive issues involves toilet seats. Should you leave the seat “up” or “down”? This also affects productivity, particularly at large companies. Hours each week are lost when employees need to adjust toilet seats. Your task is to analyze the impact different bathroom policies will have on the number of seat adjustments required.



Photo by Henry Stern

The classical assumption is that a male usually uses a toilet with the seat “up” whereas a female usually uses it with the seat “down”. However, we will divide the population into those who prefer the seat up and those who prefer it down, regardless of gender.

Now, there are several possible policies that one could use, here are a few:

1. When you leave, always leave the seat up
2. When you leave, always leave the seat down
3. When you leave, always leave the seat as you would like to find it

So, a person may have to adjust the seat prior to using the toilet and, depending on policy, may need to adjust it before leaving.

Task



Your task is to evaluate these different policies. For a given sequence of people's preferences, you are supposed to calculate how many seat adjustments are made for each policy.

Input

The first and only line of input contains a string of characters 'U' and 'D', indicating that a person in the sequence wants the seat *up* or *down*. The string has length at least 2 and at most 1000.

The first character indicates the initial position of the toilet seat, and the following $n - 1$ characters indicate how a sequence of $n - 1$ people prefer the seat. You should compute the total number of seat adjustments needed for each of the three policies described above.

Output

Output three numbers, each on a separate line, the total number of seat adjustments for each policy.

Sample Input 1

```
UUUDDUDU
```

Sample Output 1

```
6
7
4
```

Rock-Paper-Scissors Tournament

CPU TIME LIMIT

1 second

MEMORY LIMIT

1024 MB

DIFFICULTY

medium (3.3)

Rock-Paper-Scissors is game for two players, A and B , who each choose, independently of the other, one of rock, paper, or scissors. A player choosing paper wins over a player choosing rock; a player choosing scissors wins over a player choosing paper; a player choosing rock wins over a player choosing scissors. A player choosing the same thing as the other player neither wins nor loses.



A tournament has been organized in which each of n players plays k rock-scissors-paper games with each of the other players, so there are $kn(n-1)/2$ games in total. Your job is to compute the win average for each player, defined as $w/(w+l)$ where w is the number of games won, and l is the number of games lost, by the player.

Input

Input consists of several test cases. The first line of input for each case contains $1 \leq n \leq 100, 1 \leq k \leq 100$ as defined above. For each game, a line follows containing p_1, m_1, p_2, m_2 . $1 \leq p_1 \leq n$ and $1 \leq p_2 \leq n$ are distinct integers identifying two players; m_1 and m_2 are their respective moves ("rock", "scissors", or "paper"). A line containing 0 follows the last test case.

Output

Output one line each for player 1, player 2, and so on, through player n , giving the player's win average rounded to three decimal places. If the win average is undefined, output "-". Output an empty line between cases.



Sample Input 1

```
2 4
1 rock 2 paper
1 scissors 2 paper
1 rock 2 rock
2 rock 1 scissors
2 1
1 rock 2 paper
0
```

Sample Output 1

```
0.333
0.667

0.000
1.000
```



Prime Sieve

CPU TIME LIMIT

2 seconds

MEMORY LIMIT

50 MB

DIFFICULTY

medium (4.7)

Input

The first line of input consists of two integers n, q , where $1 \leq n \leq 10^8$ and $1 \leq q \leq 20000$. Then follow q lines, each containing an integer x satisfying $1 \leq x \leq n$.

Output

On the first line of output, write one line giving the number of prime numbers less than or equal to n . Then for each query x , output 1 if x is a prime and output 0 if x is composite.

Sample Input 1

```
9973 6
1
2
3
4
9972
9973
```

Sample Output 1

```
1229
0
1
1
0
0
1
```

Coven Complications

CPU TIME LIMIT

3 seconds

MEMORY LIMIT

1024 MB

DIFFICULTY

medium (3.5)

After some unexpected circumstances, you have become the ruler of the most prestigious witch coven in the forest. This coven controls some of the witch communities in the forest – specifically, the ones with no ferrets. The communities are linked by portals: each portal links exactly two communities.

Unfortunately, all communities owned by rival witch covens are swarmed with Ferret Potion Creators (FPCs). Each community i controlled by a rival contains $f_i > 0$ ferrets, and each day, the ferrets of this community create potions that drain power from f_i witches of each linked community you own.

After years of studying spells, you have finally learned a spell that can permanently protect a community from all FPC attacks. However, the spell is exhausting and can only be cast once per day. Each day proceeds as follows:

1. You cast the sealing spell on one of your unsealed communities. Witches in this community are immediately protected and lose no power from this day onwards.
2. Each of your own remaining unsealed communities loses a number of witches equal to the total number of ferrets in their directly linked rival communities.

Calculate the minimum number of witches that will lose their power before all of your communities are sealed, so you know how many shamans to hire to heal them. You may assume that each of your communities has an unlimited supply of witches.

Input



Do not get fooled by their cute appearance, the FPCs are here to drain power from your witches!

Ferret: CC BY-SA 4.0 by Alfredo

Gutiérrez on [Wikimedia](#)

[Commons](#) Hat: CC BY-SA 3.0 by

WarX on [Wikimedia Commons](#)



The input consists of:

- One line with two integers n and m ($1 \leq n \leq 10^5$, $0 \leq m \leq 10^5$), the number of communities and the number of portals between communities.
- One line with n integers $f_1, \dots, f_n$ ($0 \leq f_i \leq 10^9$ for each i), the number of ferrets in each community. Your witch coven owns a community if and only if $f_i = 0$.
- m lines, each with two integers u and v ($1 \leq u < v \leq n$), indicating that a portal links the two communities u and v . There is at most one portal between the same two communities. There are no portals from a community to itself.

Output

Output the minimum number of witches that lose their power before all of your communities are sealed.

Sample Input 1

```
5 4
0 0 10 0 5
1 2
2 3
3 4
4 5
```

Sample Output 1

```
10
```



Sample Input 2

```
7 9
20 15 0 2 0 10 0
1 2
1 4
1 5
2 7
3 4
3 5
3 6
5 6
6 7
```

Sample Output 2

```
49
```

Multi-Touch Gesture Classification

CPU TIME LIMIT

1 second

MEMORY LIMIT

1024 MB

DIFFICULTY

hard (5.9)

With the advent of touch-screen based interfaces, software designers have had to reinvent computer control for finger-based gestures. The most basic task in interpreting a gesture is classifying the touches into one of several possible gestures. This can be difficult because the touches are really blobs of pixels on the screen (not unique points), and gestures involve movement.

We define a *touch* as a connected set of pixels. A pair of pixels is part of the same touch if they are horizontally or vertically adjacent. The *touch point* is the position of a touch. It's defined as the average location of all pixels that are in the set.

A *grip* is the set of all touch points for an image. The *grip point* is the average location of all touch points in the grip. For each touch point we define its *touch vector* as the vector from the grip point to the touch point. A *gesture* is a pair of grips (initial and final), and both grips have the same number of touches. There is a one-to-one correspondence between touches in the initial and final grip. The correspondence is chosen such that the sum of squared distances between corresponding touch points is minimized.

There are three kinds of gestures: pan, zoom, and rotation. There is a distance associated with each kind. The *pan distance* is measured between the two grip points. The *grip spread* is the average distance between each touch point and the grip point. The *zoom distance* is the difference between the two grip spreads. The *touch rotation* is the signed angle (in $[-\pi, \pi]$ radians) between the two touch vectors for corresponding touch points. The *grip rotation* is the average touch rotation. The *rotation distance* is the arc length of the grip rotation along a circle with radius equal the grip spread of the initial grip.

You need to classify the gestures based whether it is a pan, zoom, or rotation. Pick the one which has the greatest associated distance.



Input

Input is one pair of images, given side-by-side. Both images are 15×30 pixels, and there will be a single space on each line separating the two images. Each image uses X to represent pixels of a touch, and a period (.) to represent no touch. Both images have the same number of touches, between 1 and 5.

Output

Print the number of touches and the type of gesture (pan, zoom, or rotate) as well as the direction of the gesture for zoom or rotate. The direction for a zoom is in if the grip spread of the final grip is smaller than that of the initial grip, otherwise it is out. The direction of a rotate is either clockwise or counter-clockwise. All distance comparisons that affect the output will be robust against numerical error of 10^{-5} .

Sample Output 1

1 pan

Sample Input 2

```
.....
.....
.....
.....
.....
.....XX.....
.....
.....XXX.....XXXX.....
.....
.....XXXX.....X.....
.....XXX.....XXX.....
.....X.....
.....XXX.....XX.....
.....
.....
.....
.....XX.....
.....
.....XXX.....
.....
.....
.....
.....
.....XX.....
.....
.....XXXX.....
.....
.....XX.....
.....
.....
```

Sample Output 2

```
3 zoom out
```



Sample Input 3

Sample Output 3



4 rotate counter-clockwise

```
.....
.....
.....
.....
.....XX.....
.....
.....XX.....
.....XXXXX.....
.....
.....XXX.....
.....
.....XXX.....XXX.....
.....XX.....
.....XX.....XXX.....
.....XXX.....
.....
.....
.....
.....
.....XXX.....
.....XXX.....
.....XX.....XXXX.....
.....X.....
.....XXXX.....
.....
.....
.....
.....
```



Antiarithmetic?

CPU TIME LIMIT

1 second

MEMORY LIMIT

1024 MB

DIFFICULTY

hard (6.5)

A sequence $a_1, a_2, a_3, \dots$ is an arithmetic progression if there exists a constant c such that $a_i - a_{i+1} = c$ for $i = 1, 2, 3, \dots$

A permutation of n is a bijective function of the initial n natural numbers: $0, 1, \dots, n - 1$. A permutation p is called antiarithmetic if there is no subsequence of it forming an arithmetic progression of length greater than 2, i.e., there are no three indices $0 \leq i < j < k < n$ such that (p_i, p_j, p_k) forms an arithmetic progression.

For example, the sequence $(2, 0, 1, 4, 3)$ is an antiarithmetic permutation of 5. The sequence $(0, 5, 4, 3, 1, 2)$ is not an antiarithmetic permutation as its first, fifth and sixth term $(0, 1, 2)$ form an arithmetic progression; and so do its second, forth and fifth term $(5, 3, 1)$.

Your task is to check whether a given permutation of n is antiarithmetic.

Input

There are several test cases (at most 5), followed by a line containing 0. Each test case is a line of the input file containing a natural number $3 \leq n \leq 10\,000$ followed by a colon and then followed by n distinct numbers separated by whitespace. All n numbers are nonnegative integers smaller than n .

Output

For each test case output one line with yes or no stating whether the permutation is antiarithmetic or not.



Sample Input 1

```
3: 0 2 1
5: 2 0 1 3 4
6: 2 4 3 5 0 1
0
```

Sample Output 1

```
yes
no
yes
```

House Deconstruction

CPU TIME LIMIT

1 second

MEMORY LIMIT

2048 MB

DIFFICULTY

hard (8.1)

In the land of Circleland, there is a circle that has equally spaced points around its circumference. The distance between any two adjacent points is 1.

There are people and houses on the circle's points. Each point contains a person, an empty house, or nothing at all. Each person would like to walk to a different house. Each house can contain at most one person. People can only walk along the circumference of the circle; they cannot cut across.

Currently, there are more houses than people, so you'd like to destroy some of the houses. Suppose you destroy a set of houses S . Let $f(S)$ be the minimum total amount of walking needed to get each person to a different non-destroyed house.

Compute the minimum value of $f(S)$, compute how many sets of houses S achieve this minimum value. Since the number of sets S can be large, output it modulo 998 244 353.

Input

The first line of input contains three integers x , n , and m ($1 \leq n < m \leq 2 \cdot 10^5, n + m \leq x \leq 10^9$), where x is the number of points around the circle, n is the number of people, and m is the number of houses. The points are labeled 1 to x , and point x is adjacent to point 1.

The next $n + m$ lines each contain two tokens, an integer p ($1 \leq p \leq x$) and a character t ($t \in \{\text{P}, \text{H}\}$), where p denotes the position of a person or house, and t denotes the type of the point, either P for person or H for house. All positions are distinct, and the positions will be given in increasing order.

Output

Output two lines. On the first line output the minimum possible value of $f(S)$. On the second line output the number of sets S that achieve this minimum value, modulo 998 244 353.

Sample Explanation

For the first sample, the minimum total walking distance is 2. We can destroy the set of houses at $\{2, 5\}$, $\{4, 5\}$ or $\{5, 6\}$.

For the second sample, we can destroy the set of houses at $\{6, 31415926\}$ for a minimum total walking distance of 4. Note that even though the minimum walking distance can be achieved in multiple ways, it is only counted once since the set of destroyed houses is the same.

Sample Input 1

```
6 2 4
1 P
2 H
3 P
4 H
5 H
6 H
```

Sample Output 1

```
2
3
```

Sample Input 2

```
10000000000 2 4
1 P
6 H
31415926 H
999999998 H
999999999 H
10000000000 P
```

Sample Output 2

```
4
1
```

Intellectual Property

CPU TIME LIMIT

1 second

MEMORY LIMIT

1024 MB

DIFFICULTY

hard (8.9)

TDP Inc. has decided to sue JCN Inc. for copyright infringement. To this end, TDP wishes to find *infringing segments* within JCN's code base, to show to selected media representatives. Since TDP has fired all its technical staff, it is looking to hire a consultant to be paid on a contingency basis should the lawsuit be successful. To demonstrate your qualifications for this position, you are to solve the problem on a number of test cases.



Input

Each test case begins with a positive integer $k \leq 1000$, the number of infringing segments to be found. Following this line are two code bases. The first code base is preceded by the line "BEGIN TDP CODEBASE" and contains a number of lines followed by the line "END TDP CODEBASE". The second code base is preceded by "BEGIN JCN CODEBASE" and followed by "END JCN CODEBASE". The line "END TDP CODEBASE" does not appear in the first code base and the line "END JCN CODEBASE" does not appear in the second. Each code base consists only of printable ASCII characters (specifically ASCII codes 32–126) and newlines. A line containing 0 follows the last test case.

Output

For each test case you should output:

1. a line containing "CASE n " where n is the number of the test case;



2. up to k infringing segments. Each segment should be printed exactly as it appears in the JCN code base (including newlines and whitespace) and should be preceded by a line containing “INFRINGING SEGMENT m LENGTH l POSITION p ” where m is the number of the segment within the test case, l is the length of the segment (in characters) and p is the position of the segment (in characters from the start of the JCN codebase). Output an empty line between test cases.

A code base is simply a string of characters. An infringing segment is a non-empty contiguous sequence of characters in the JCN code base that is textually identical to some contiguous sequence of characters in the TDP code base, and is not contained in a larger infringing segment. You should consider all characters in the code base, including spaces and the *newline* character at the end of each line.

If there are k or fewer common segments, print them all in decreasing order by length; if several segments have the same length, print them in the order they occur in JCN’s code base. If there are more than k segments, print the first k according to the given order.

You may assume that no code base contains more than 50 000 characters.

Sample Input 1

```
6
BEGIN TDP CODEBASE
the quick brown fox
jumps over the lazy dog.
so there!
END TDP CODEBASE
BEGIN JCN CODEBASE
now is the time for all
good men to come to the aid
of the party.
so there!
END JCN CODEBASE
100
BEGIN TDP CODEBASE
xyzzzy
END TDP CODEBASE
BEGIN JCN CODEBASE
xyzzabczzyy
END JCN CODEBASE
0
```

Sample Output 1

```
CASE 1
INFRINGING SEGMENT 1 LENGTH 12
POSITION 64
.
so there!

INFRINGING SEGMENT 2 LENGTH 5
POSITION 6
the
INFRINGING SEGMENT 3 LENGTH 5
POSITION 42
o the
INFRINGING SEGMENT 4 LENGTH 5
POSITION 43
the
INFRINGING SEGMENT 5 LENGTH 5
POSITION 54
the
INFRINGING SEGMENT 6 LENGTH 3
POSITION 15
fo

CASE 2
INFRINGING SEGMENT 1 LENGTH 4
POSITION 0
xyzz
INFRINGING SEGMENT 2 LENGTH 3
POSITION 7
zzy
INFRINGING SEGMENT 3 LENGTH 2
POSITION 10
```



y

Office Space

CPU TIME LIMIT

3 seconds

MEMORY LIMIT

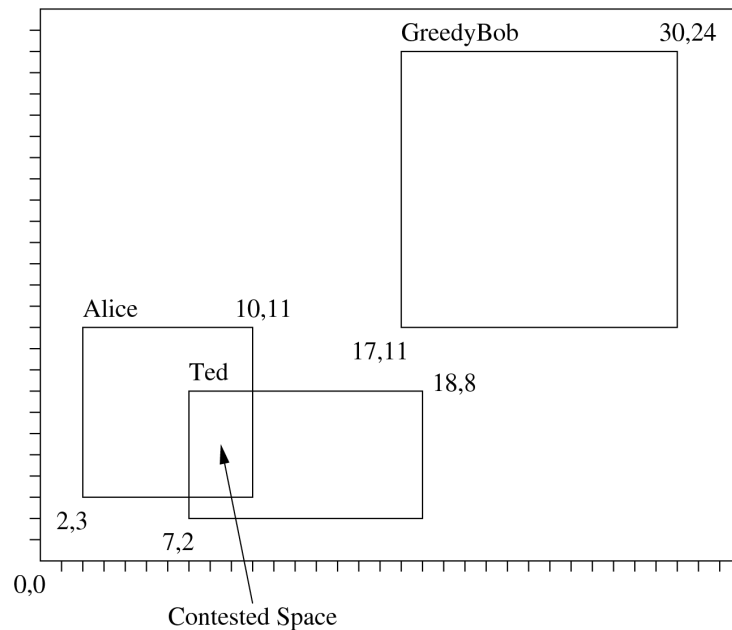
1024 MB

DIFFICULTY

medium (3.1)

Your company is moving to a new, larger office building. The new office is a rectangular space that will eventually be populated with cubicles. Your employees want to request particular positions for their cubicles, so you are setting up a system that lets them make these requests. To help automate this process, you have expressed the new office building using a coordinate system where each unit is one foot. The south-west corner of the new building's floor space is assigned coordinate $(0, 0)$, the positive X axis is aligned with the inner edge of the building's south wall and the positive Y axis is aligned with the west wall. Employees request a position for their cubicle by giving the coordinates of the cubicle's south-west corner and its north-east corner.

You don't expect this technique for dividing up the space to work very well the first time around. Some space may not get allocated and space may be requested by more than one employee. Your job is to compile a report of how well these requests would allocate the office space and how much contention there is between requests.



Input

Input consists of up to 10 test cases. Each case starts with a line containing a pair of integers w and h giving the size of the new office space (w is the number of feet west-to-east; h is the number of feet south-to-north). Both of these numbers are in the range $1 \dots 100$. After this is a line containing an integer $0 \leq n \leq 20$ giving the number of employees you have. Following this are n cubicle placement requests, one per line. Each request starts with the name of the employee. The name is a string of 1 to 20 lower- and/or upper-case letters (a-z). The name is followed by four integers x_1, y_1, x_2, y_2 where (x_1, y_1) indicate the coordinates of the south-west corner of their desired cubicle placement and (x_2, y_2) indicate the coordinates for the north-east corner. Each set of request coordinates satisfies $0 \leq x_1 \leq x_2 \leq w$ and $0 \leq y_1 \leq y_2 \leq h$. The sequence of test cases ends at the end of file.

Output

For each test case, print out a report that starts with the total number of square feet in the building and the number of square feet that no employee has requested (the unallocated space). Next, give the total number of square feet that are contested because more than one employee has requested the same region of the floor. Finally, for each employee give the number square feet that that employee can be guaranteed to have. This is the total area that they requested minus any regions that were also requested by another employee. List the

employees in the same order they were given in the input. Leave a blank line after the output for each test case.

Sample Input 1

```
33 26
3
Alice 2 3 10 11
Ted 7 2 18 8
GreedyBob 17 11 30 24
20 10
2
Employee1 0 0 9 10
Employee2 11 0 20 10
```

Sample Output 1

```
Total 858
Unallocated 574
Contested 15
Alice 49
Ted 51
GreedyBob 169

Total 200
Unallocated 20
Contested 0
Employee1 90
Employee2 90
```


A Mazing!

CPU TIME LIMIT

1 second

MEMORY LIMIT

1024 MB

DIFFICULTY

hard (5.6)

A maze consists of a collection of equal sized square cells, where any or all of the sides may be a wall or a door. The maze may have no exit or multiple exits. Cells are typically arranged so that they may share sides with other cells as shown in the four sample mazes below:

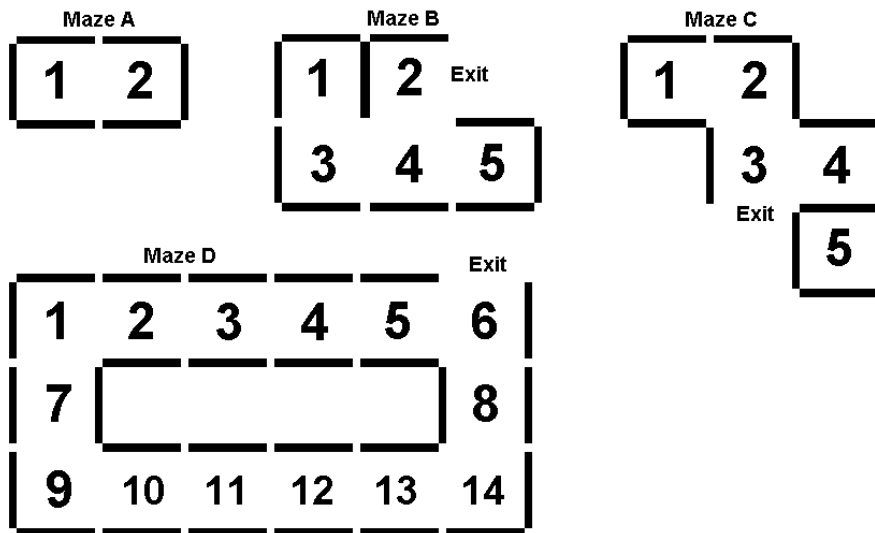
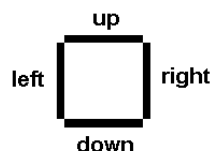


Figure 1: (The numbers in the cells above are for illustrative purposes only.)

Each side of a cell is labeled with a direction to allow navigation:



For each maze, the starting point is someplace in the maze (for example, any of the numbered cells in the samples above). In the samples above:



- Maze A has no way out.
- Maze B has an exit (solution) to the right of cell 2.
- Maze C has an exit down from cell 3, unless the starting point is cell 5, in which case there is no way out
- Maze D has an exit up from cell 6.

For example, using Maze D above, if the starting point is cell 9, one possible set of directions to get to the exit would be: right, right, right, right, right, up, up, up.

For this problem, you will write a program that finds an exit to a maze. Your program must operate *interactively*. That is, your program will make a move by providing a direction (right, down, left or up), and the judging software will send back one of four responses:

1. **wall** - indicates that a wall is there and you cannot proceed in that direction
2. **ok** - indicates that there is door there and you may proceed in that direction to the neighboring cell.
3. **solved** - indicates that you have successfully found an exit to the maze.
4. **wrong** - indicates that your program made an error, as discussed below.

If your program determines there is no way out of the maze, you should send the precise string “no way out” (without the quotes) instead of a direction. If there is in fact no way out of the maze, you will receive a **solved** reply.

Your program will receive a **wrong** indication if any of the following occur:

1. Your program sends “no way out”, even though there is a way out.
2. Your program makes the same move (direction) from the same cell twice.

After receiving a **wrong** or a **solved** reply, your program should exit.

Input/Output

This is an interactive program. The input you receive is a function of the output you generate. All input and output strings must end in a new-line character. You should never send extra blank lines.

You must make sure that your program's standard output stream is flushed after you output a the new-line character that completes a command. This is accomplished with `System.out.flush()` in Java, `stdout.flush()` in Python, `fflush(stdout)` in C, and `cout << flush` in C++.

The first thing your program must do when it starts up is to send its first move (up, down, right or left), followed by a new-line character. It will then wait for a new-line terminated response on the standard input. The response will be one of **wall**, **ok**, **solved**, or **wrong** indication. Your program will then make another move based on the response it received as discussed above. This process will repeat until your program receives a **wrong** or **solved** indication.

Example (User output in Teletype, Computer judge output in **Bold**). (This sample run has no relationship to the samples shown above).



Sample Run
down
wall
right
wall
left
wall
up
ok
right
ok
down
ok



right

wall

left

wall

up

ok

right

solved

It is guaranteed that the maze will not be larger than 100×100 in any dimension.